

```
-- =====
-- Database Objects for 2afashtak (Lost & Found)
-- Follows Database Design, SQL Queries, and Stored Procedures/Triggers requirements
-- =====

-- 1. DATABASE DESIGN + NORMALIZATION (3NF)

CREATE DATABASE LostAndFoundDB;
GO
USE LostAndFoundDB;
GO

-- Table: Users
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    FullName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) UNIQUE NOT NULL,
    PhoneNumber NVARCHAR(20),
    CreatedAt DATETIME DEFAULT GETDATE()
);

-- Table: Categories
CREATE TABLE Categories (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(50) NOT NULL UNIQUE
);

-- Table: Locations
CREATE TABLE Locations (
    LocationID INT PRIMARY KEY IDENTITY(1,1),
    LocationName NVARCHAR(100) NOT NULL,
    AddressDescription NVARCHAR(255)
);

-- Table: Items (Lost/Found Reports)
-- Normalized: References Users, Categories, and Locations
CREATE TABLE Items (
    ItemID INT PRIMARY KEY IDENTITY(1,1),
    UserID INT NOT NULL,
    CategoryID INT NOT NULL,
    LocationID INT NOT NULL,
    Title NVARCHAR(150) NOT NULL,
    Description NVARCHAR(MAX),
    ItemType NVARCHAR(10) CHECK (ItemType IN ('Lost', 'Found')),
    DateReported DATE DEFAULT CAST(GETDATE() AS DATE),
    Status NVARCHAR(20) DEFAULT 'Active' CHECK (Status IN ('Active', 'Resolved', 'Cancelled')),
    Timestamp DATETIME DEFAULT GETDATE(),

    CONSTRAINT FK_Items_Users FOREIGN KEY (UserID) REFERENCES Users(UserID),
    CONSTRAINT FK_Items_Categories FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID),
    CONSTRAINT FK_Items_Locations FOREIGN KEY (LocationID) REFERENCES Locations(LocationID)
);

-- Table: Matches
CREATE TABLE Matches (
    MatchID INT PRIMARY KEY IDENTITY(1,1),
    LostItemID INT NOT NULL,
    FoundItemID INT NOT NULL,
    MatchDate DATETIME DEFAULT GETDATE(),
    ConfidenceScore DECIMAL(5,2), -- e.g. 0.00 to 100.00
```

```
        CONSTRAINT FK_Matches_Lost FOREIGN KEY (LostItemID) REFERENCES Items(ItemID),
        CONSTRAINT FK_Matches_Found FOREIGN KEY (FoundItemID) REFERENCES Items(ItemID)
    );

-- Table: AuditLog
CREATE TABLE AuditLog (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    ItemID INT,
    OldStatus NVARCHAR(20),
    NewStatus NVARCHAR(20),
    ChangedDate DATETIME DEFAULT GETDATE()
);

-- 2. SQL QUERIES (JOINS, Aggregation, Projection, Subqueries)

-- A. JOIN: Retrieve all active items with their full category and location details
SELECT
    i.ItemID,
    i.Title,
    c.CategoryName,
    l.LocationName,
    u.FullName AS ReportedBy,
    i.ItemType,
    i.DateReported
FROM Items i
JOIN Categories c ON i.CategoryID = c.CategoryID
JOIN Locations l ON i.LocationID = l.LocationID
JOIN Users u ON i.UserID = u.UserID
WHERE i.Status = 'Active';

-- B. AGGREGATION: Count items per category to show on a chart
SELECT
    c.CategoryName,
    COUNT(i.ItemID) AS TotalItems,
    SUM(CASE WHEN i.ItemType = 'Lost' THEN 1 ELSE 0 END) AS LostCount,
    SUM(CASE WHEN i.ItemType = 'Found' THEN 1 ELSE 0 END) AS FoundCount
FROM Categories c
LEFT JOIN Items i ON c.CategoryID = i.CategoryID
GROUP BY c.CategoryName;

-- C. SUBQUERY: Find users who have reported more than one lost item
SELECT FullName, Email
FROM Users
WHERE UserID IN (
    SELECT UserID
    FROM Items
    WHERE ItemType = 'Lost'
    GROUP BY UserID
    HAVING COUNT(ItemID) > 1
);

-- D. PROJECTION: Get a summary view for the mobile app (Specific columns only)
SELECT
    UPPER(Title) AS ItemTitle,
    LEFT(Description, 50) + '...' AS ShortDesc,
    DateReported
FROM Items
WHERE DateReported >= DATEADD(day, -7, GETDATE());

-- 3. STORED PROCEDURES + TRIGGERS
```

```
-- A. STORED PROCEDURE: Register a new item and return its ID
CREATE PROCEDURE sp_ReportItem
    @UserID INT,
    @CategoryID INT,
    @LocationID INT,
    @Title NVARCHAR(150),
    @Description NVARCHAR(MAX),
    @ItemType NVARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Items (UserID, CategoryID, LocationID, Title, Description, ItemType)
    VALUES (@UserID, @CategoryID, @LocationID, @Title, @Description, @ItemType);

    SELECT SCOPE_IDENTITY() ASNewItemID;
END;
GO

-- B. STORED PROCEDURE: Get Dashboard Stats
CREATE PROCEDURE sp_GetDashboardStats
AS
BEGIN
    SELECT
        (SELECT COUNT(*) FROM Items WHERE ItemType = 'Lost' AND Status = 'Active') AS ActiveLost,
        (SELECT COUNT(*) FROM Items WHERE ItemType = 'Found' AND Status = 'Active') AS
ActiveFound,
        (SELECT COUNT(*) FROM Items WHERE Status = 'Resolved') AS TotalResolved;
END;
GO

-- C. TRIGGER: Audit Item Status Changes
-- Automatically logs whenever an item's status is updated
CREATE TRIGGER tr_AuditItemStatus
ON Items
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Status)
    BEGIN
        INSERT INTO AuditLog (ItemID, OldStatus, NewStatus)
        SELECT d.ItemID, d.Status, i.Status
        FROM Deleted d
        JOIN Inserted i ON d.ItemID = i.ItemID;
    END
END;
GO

-- D. TRIGGER: Prevent Deleting Resolved Items
-- Ensures data integrity for resolved cases
CREATE TRIGGER tr_PreventDeleteResolved
ON Items
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (SELECT 1 FROM deleted WHERE Status = 'Resolved')
    BEGIN
        RAISERROR ('Cannot delete resolved items. Please archive them instead.', 16, 1);
    END
    ELSE
    BEGIN
        DELETE FROM Items WHERE ItemID IN (SELECT ItemID FROM deleted);
    END
END;
```

```
END
END;
GO

-- Sample Data Seeding
INSERT INTO Categories (CategoryName) VALUES ('Electronics'), ('Wallet/Purse'), ('Keys'),
('Jewelry'), ('Documents'), ('Other');
INSERT INTO Locations (LocationName) VALUES ('Central Park'), ('Main Station'), ('Airport Terminal
1'), ('Library');
INSERT INTO Users (FullName, Email, PhoneNumber) VALUES ('John Doe', 'john@example.com',
'123-456-7890');
```